Service层单函数单元测试计划（JUnit）

1. 目标与范围

本计划针对 `src/main/java/com/demo/service` 下的 Service 实现层，设计“单个函数”的单元测试方案，要求同时使用黑盒与白盒方法，并用 JUnit 编写可执行测试脚本，验证后端单元行为是否符合预期。

测试范围聚焦一个代表性函数：

- 目标函数： `OrderServiceImpl.confirmOrder(int orderID)`
- 文件位置： `src/main/java/com/demo/service/impl/OrderServiceImpl.java`

选择原因：

- 存在清晰业务分支（订单存在/不存在）
- 有明确副作用（调用 DAO 更新状态）
- 依赖边界清楚（ `OrderDao` ），适合纯单元测试（Mock）
- 可同时体现黑盒与白盒测试设计

2. 代码分析结论（service目录）

`service/impl` 中大部分方法是 DAO 透传；更适合做“有判定分支”的函数单测的包括：

- `OrderServiceImpl.confirmOrder / finishOrder / rejectOrder`
- `MessageServiceImpl.confirmMessage / rejectMessage`

其中 `confirmOrder` 逻辑如下（抽象）：

1. 通过 `orderDao.findByOrderID(orderID)` 查订单
2. 若返回 `null` ，抛出 `RuntimeException`
3. 否则调用 `orderDao.updateState(STATE_WAIT, order.getOrderID())`

3. 测试策略

3.1 黑盒测试设计（基于输入输出与业务规则）

不关心内部实现，只关注行为是否正确：

用例ID	输入	前置条件	期望结果
BB-01	`orderID=1001`	DAO中存在该订单	方法正常返回；订单状态应被更新为 `STATE_WAIT`
BB-02	`orderID=9999`	DAO中不存在该订单	抛出 `RuntimeException`
BB-03	`orderID=0/-1`	非法或不存在ID	抛出 `RuntimeException` ，且不应更新状态

说明： `confirmOrder` 本身不做参数格式校验，因此非法 ID 在当前实现里会走“不存在订单”分支。

3.2 白盒测试设计（基于分支与调用路径）

按代码结构覆盖所有分支路径：

- 路径 P1： `order == null` -> 抛异常 -> `updateState` 不执行
- 路径 P2： `order != null` -> 执行 `updateState(STATE_WAIT, orderID)`

覆盖目标：

- 语句覆盖率：目标函数 100%
- 分支覆盖率：目标函数 100%
- 关键交互断言：
  - `findByOrderID` 调用 1 次
  - 成功路径 `updateState` 调用 1 次，参数正确
  - 异常路径 `updateState` 调用 0 次

4. 测试实现方案（JUnit5 + Mockito）

当前项目已包含 `spring-boot-starter-test` ，可直接使用 JUnit5 + Mockito。

建议测试类：

- `src/test/java/com/demo/service/impl/OrderServiceImplConfirmOrderTest.java`

脚本模板（可直接作为起始版本）：

```java
package com.demo.service.impl;

import com.demo.dao.OrderDao;
import com.demo.dao.VenueDao;
import com.demo.entity.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class OrderServiceImplConfirmOrderTest {

    @Mock
    private OrderDao orderDao;

    @Mock
    private VenueDao venueDao;

    @InjectMocks
    private OrderServiceImpl orderService;

    @Test
    void confirmOrder_shouldUpdateState_whenOrderExists() {
        Order order = new Order();
        order.setOrderID(1001);
        when(orderDao.findByOrderID(1001)).thenReturn(order);

        orderService.confirmOrder(1001);

        verify(orderDao, times(1)).findByOrderID(1001);
        verify(orderDao, times(1)).updateState(2, 1001); // 2 == STATE_WAIT
        verifyNoMoreInteractions(orderDao);
    }

    @Test
    void confirmOrder_shouldThrowException_whenOrderNotExists() {
        when(orderDao.findByOrderID(9999)).thenReturn(null);
```

```java
        assertThrows(RuntimeException.class, () -> orderService.confirmOrder(9999));

        verify(orderDao, times(1)).findByOrderID(9999);
        verify(orderDao, never()).updateState(anyInt(), anyInt());
    }

    @Test
    void confirmOrder_shouldThrowException_whenOrderIdIsInvalid() {
        when(orderDao.findByOrderID(0)).thenReturn(null);

        assertThrows(RuntimeException.class, () -> orderService.confirmOrder(0));

        verify(orderDao, times(1)).findByOrderID(0);
        verify(orderDao, never()).updateState(anyInt(), anyInt());
    }
}
```

5. 执行步骤

1. 在 `src/test/java/com/demo/service/impl` 新建上述测试类
2. 按黑盒/白盒矩阵补齐测试方法
3. 执行 `mvn test`
4. 观察测试结果，确认 `confirmOrder` 的成功与异常路径都通过
5. 如需度量覆盖率，增加 JaCoCo 并检查该函数语句/分支覆盖

6. 验收标准

- 测试全部通过
- `confirmOrder` 两条逻辑路径都被覆盖
- 成功路径状态更新参数正确（`STATE_WAIT`，`orderID`）
- 异常路径不发生误更新
- 单元测试不依赖真实数据库（Mock 隔离外部依赖）

7. 后续扩展（同模式复用）

可直接复用该模式到：

- `OrderServiceImpl.finishOrder/rejectOrder`
- `MessageServiceImpl.confirmMessage/rejectMessage`

复用时仅替换：

- 目标状态值
- DAO查询方法
- 异常断言与交互断言